STACK (LIFO — Last In, First Out)

Use when: reversing order of arrival,matching brackets,undo operations,iterative DFS.

```
stack<int> st;
st.push(x);                          // add to top
st.top();                            // peek top (doesn't remove)
st.pop();                            // remove top
st.empty();  st.size();
```

```
// Check if brackets are balanced
stack<char> st;
bool valid = true;
for (char c : s) {
    if (c == '(') st.push(c);
    else if (c == ')') {
        if (st.empty()) { valid = false; break; }
        st.pop();
    }
}
if (!st.empty()) valid = false;    // unmatched opening brackets left
```

QUEUE (FIFO — First In, First Out)

Use when: processing in arrival order — BFS is the main use case.

```
queue<int> q;
q.push(x);                           // add to back
q.front();                           // peek front
q.back();                            // peek back
q.pop();                             // remove front
q.empty();  q.size();
```

// Basic BFS traversal

```
queue<int> q;
vector<bool> visited(n, false);
q.push(start);
visited[start] = true;

while (!q.empty()) {
    int cur = q.front(); q.pop();
    for (int next : adj[cur]) {
        if (!visited[next]) {
            visited[next] = true;
            q.push(next);
        }
    }
}
```

DEQUE (double-ended queue — useful for sliding window)

```
deque<int> dq;
dq.push_back(x);   dq.push_front(x);
dq.pop_back();     dq.pop_front();
dq.front();        dq.back();
dq.empty();        dq.size();
```